UD2 – Persistència en BD relacionals mitjançant JDBC



ÍNDEX

1 Introducció....4
1.1 ODBC....4
1.1.1 L'arribada dels estàndards....5
1.1.2 La situació actual....6
2 JDBC....7
2.1 Arquitectura JDBC....8
2.2 Funcions de JDBC....9
3 Descripció general del API JDBC....10
4 Treballar amb JDBC en un projecte....12
5 Connectar a BBDD amb JDBC usant DriverManager....14
6 Connectar a BBDD amb JDBC usant DataSource....18
6.1 Tipus de DataSource....18
6.2 Pool de connexions....18
6.3 apache dbcp: BasicDataSource....19
6.4 Fitxer de propietats....20
7 La interfície *Statement*....23
8 ResultSet....28
8.1 Necessitat d'alliberar els recursos....29
8.2 Tipus de ResultSet....31
8.3 Obtenint dades....32
8.4 Tipus de dades i conversions....33

8.5 Desplaçaments....36
8.6 Mètodes UpdateXXX ()....39
8.7 Mètodes insertRow() i deleteRow()....41
9 Objectes PreparedStatement....43
9.1 Utilització paràmetres d'entrada....44
10 Transaccions....46
11 Maneig d'errors en JDBC....48
11.1 SqlException....49
11.2 SQLWarning....51
11.3 DataTruncation....51
12 DatabaseMetadata....53
12.1 Obtenint informació d'una Base de dades....53
13 ResultSetMetadata....60
14 Objectes CallableStatement....62
14.1 Utilització de paràmetres....62
15 JDBC Batch Processing....64
15.1 BatchUpdateException....65

1 Introducció

La història de les bases de dades relacionals ha transcorregut íntimament lligada a la història de les aplicacions d'arquitectura distribuïda i en particular, arquitectures client-servidor.

Fa poc més d'un quart de segle, la persistència de les aplicacions formava part del desenvolupament d'estes, que es concebien com un tot monolític. L'emmagatzematge i la recuperació de les dades es mesclaven i difuminaven amb l'explotació d'estes. Les bases de dades solien disposar de llenguatges de programació propis que incorporaven crides i sentències específiques d'accés a les dades.

En realitat, esta situació no representava molts avantatges ni per a les empreses desenvolupadores dels SGBD ni per a les empreses usuàries:

- Les primeres es trobaven que mantindre el desenvolupament d'un llenguatge de programació resultava realment costós si no es volia quedar ràpidament desfasat.
- Les empreses usuàries, d'altra banda, es trobaven lligades a un llenguatge de programació que no sempre els donava resposta a totes les seues exigències. A més, plantejar-se qualsevol canvi de sistema gestor de dades representava una meta impossible, atés que implicava haver de programar de nou totes les aplicacions de l'empresa.

Calia desvincular els llenguatges de desenvolupament dels SGBD.

1.1 ODBC

A mesura que les teories de dades relacionals anaven agafant força i les xarxes guanyaven adeptes gràcies a l'increment de l'eficiència a preus realment competitius, van començar a implementar-se uns sistemes gestors de bases de dades basats en la **tecnologia client-servidor**.

La tecnologia client-servidor va permetre **aïllar les dades i els programes** específics d'accés a estes, del desenvolupament de l'aplicació. La raó principal d'esta divisió va ser segurament possibilitar l'accés remot a les dades de qualsevol ordinador connectat a la xarxa. La veritat, no obstant això, és que este fet va espentar els sistemes de bases de dades a desenvolupar-se d'una forma aïllada i a crear protocols i llenguatges específics per a poder-se comunicar remotament amb les aplicacions que corrien en ordinadors externs.

A poc a poc, el programari al voltant de les bases de dades va créixer espectacularment intentant donar resposta a un màxim ventall de demandes a través de sistemes altament configurables. És el que hui dia es coneix com *middleware* o capa intermèdia de persistència. És a dir, el conjunt d'aplicacions, utilitats, biblioteques, protocols i llenguatges, situats tant en la part servidor com en la part client, que permeten connectar-se remotament a una base de dades per a configurar-ho o explotar les seues dades.

1.1.1 L'arribada dels estàndards

Inicialment, cada empresa desenvolupadora d'un SGBD implementava solucions propietàries específiques per al seu sistema, però prompte es van adonar que col·laborant conjuntament podien traure major rendiment i avançar molt més ràpidament.

Sostenint-se en la teoria relacional i en algunes implementacions primerenques de les empreses IBM i Oracle, es va desenvolupar el llenguatge de consulta de dades anomenat SQL. Este repte va suposar, sens dubte, un gran pas, però les aplicacions necessitaven API amb funcions que permeteren fer crides des de llenguatge de desenvolupament per a enviar les consultes realitzades amb el SQL.

Cada SGBD té la seua pròpia connexió i el seu propi API.

El grup anomenat SQL Access Group, en el qual participaven prestigioses empreses del sector com Oracle, Informix, Ingres, DEC, Sun o HP, va definir una API universal amb independència del llenguatge de desenvolupament i la base de dades a connectar.

En 1992, Microsoft i Simba implementen el ODBC (Open Data Base Connectivity), una API basada en la definició de SQL Access Group, que s'integra en el sistema operatiu de Windows i que permet afegir múltiples connectors a diverses

bases de dades SQL de forma molt senzilla i transparent, ja que els connectors són autoinstal·lables i totalment configurables des de les mateixes ferramentes del sistema operatiu.

L'arribada de l'ODBC va representar un avanç sense precedents en el camí cap a la interoperabilitat entre bases de dades i llenguatges de programació. La majoria d'empreses desenvolupadores de SGBD van incorporar els drivers de connectivitat a les utilitats dels seus sistemes i els llenguatges de programació més importants van desenvolupar biblioteques específiques per a suportar l'API ODBC.

1.1.2 La situació actual

Actualment, ODBC continua sent una iniciativa adequada de connexió a SGBD relacionals. El seu desenvolupament seguix liderat per Microsoft, però existixen versions en altres sistemes operatius aliens a la companyia com UNIX / LINUX o MAC. Els llenguatges més populars de desenvolupament mantenen actualitzades les biblioteques de comunicació amb les successives versions que han anat apareixent i la majoria de SGBD disposa d'un controlador ODBC bàsic.

2 JDBC



Quasi de manera simultània a ODBC, l'empresa Sun Microsystems, en 1997 va traure a la llum JDBC, una API connector de bases de dades, implementat específicament per a usar amb el llenguatge Java.

Es tracta d'una API bastant similar a ODBC quant a funcionalitat, però adaptat a les especificitats de Java. És a dir, la funcionalitat es troba encapsulada en classes (ja que Java és un llenguatge totalment orientat a objectes) i a més, no depén de cap plataforma específica, d'acord amb la característica multiplataforma defensada per Java.

Este connector serà l'API que treballarem detalladament en esta unitat, ja que Java no disposa de cap biblioteca específica ODBC. Les raons esgrimides per Sun són que ODBC no es pot utilitzar directament a Java ja que està implementat en C i no és orientat a objectes. En altres paraules, usar ODBC des del llenguatge Java necessitaria en qualsevol cas una biblioteca que adaptara el API ODBC als requeriments Java.

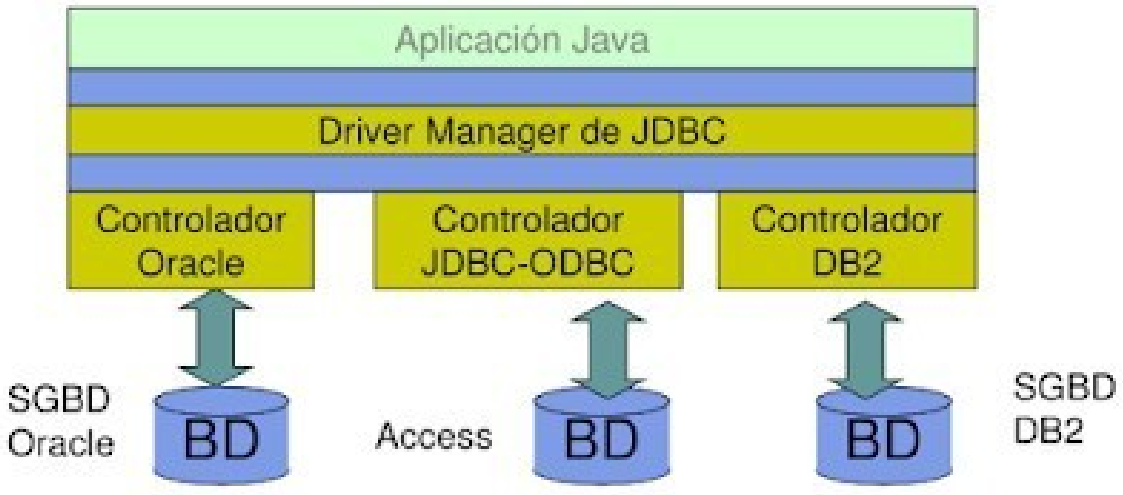
Sun Microsystems va optar per una solució que permet fer totes dues coses alhora:

- D'una banda ha implementat un **connector específic de Java** que pot comunicar-se directament amb qualsevol base de dades usant controladors (drivers) de manera molt similar a com es fa en ODBC.
- Per l'altre costat, incorpora de sèrie un **driver especial que actua d'adaptador entre l'especificació JDBC i l'especificació ODBC**. Este controlador se sol cridar també **pont** (bridge en anglés**) JDBC-ODBC**. Usant este driver podrem enllaçar qualsevol aplicació Java amb qualsevol connexió ODBC.

Actualment, la gran majoria de SGBD disposa de drivers JDBC, però en cas d'haver de treballar amb un sistema que no tinga, si disposa de controlador *ODBC, podrem utilitzar el pont JDBC-ODBC per a aconseguir la connexió des de Java.

2.1 Arquitectura JDBC

Igual que ODBC, cada desenvolupador de sistemes gestors de bases de dades implementa els seus propis controladors JDBC. Per a aconseguir la interoperabilitat entre controladors, la biblioteca estàndard JDBC conté un gran nombre d'interfícies sense les classes que implementen.



Cada controlador de cada fabricant incorpora les classes que implementen les interfícies de la API JDBC. D'esta manera, el controlador utilitzat serà totalment transparent a l'aplicació, és a dir, durant el desenvolupament de l'aplicació no serà necessari saber quin serà el driver que finalment s'utilitzarà per a l'explotació de les dades, ja que l'aplicació declararà exclusivament les interfícies de l'API JDBC.

D'esta manera s'aconseguix independitzar l'aplicació dels controladors permetent la substitució del controlador original per qualsevol altre compatible JDBC sense pràcticament necessitat d'haver de modificar el codi de l'aplicació.

JDBC és una especificació formada per una col·lecció d'interfícies i classes abstractes, que han d'implementar tots els fabricants de drivers que vulguen realitzar una implementació de la seua driver 100% Java i compatible amb JDBC (JDBC-compliant driver).

2.2 Funcions de JDBC

Bàsicament l'API JDBC fa possible la realització de les següents tasques:

- **Establir una connexió** amb la base de dades.
- **Enviar sentències a la base de dades** (SQL, manipulació de dades, creació de taules, permisos, etc.)
- **Processar els resultats** de l'execució de les sentències.

3 Descripció general del API JDBC.

Dins de l'API de JDBC existixen dos conjunts d'interfícies: d'una banda estan aquelles interfícies utilitzades per programadors d'aplicacions que utilitzen l'accés a bases de dades (este és el nostre cas i és en el qual ens centrarem) i d'altra banda existix una part de l'API de JDBC que és d'un nivell més baix i que és utilitzat pels programadors de drivers.

D'esta enumeració d'interfícies, classes i excepcions les més importants són:

- **java.sql.DriverManager:** és la classe gestora dels drivers. Esta classe s'encarrega de carregar i seleccionar el driver adequat per a realitzar la connexió amb una base de dades determinada.
- **java.sql.Connection**: representa una connexió amb una base de dades.
- **java.sql.Statement**: actua com un contenidor per a executar sentències SQL sobre una base de dades. Esta interfície té altres dos subtipus:
  - java.sql.PreparedStatement: per a l'execució de sentències SQL precompilades a les quals se li poden passar paràmetres d'entrada.
  - java.sql.CallableStatement: que permet executar procediments emmagatzemats d'una base de dades.
- **java.sql.ResultSet**: controla l'accés als resultats de l'execució d'una consulta, és a dir, d'un objecte *Statement*. Permet també la modificació d'estos resultats.
- **java.sql.SqlException**: per a tractar les excepcions que es produïsquen en manipular la base de dades, ja siga durant el procés de connexió, desconnexió o obtenció i modificació de les dades.
- **java.sql.BatchUpdateException**: excepció que es llançarà quan es produïsca algun error a l'hora d'executar actualitzacions en batch sobre la base de dades.
- **java.sql.Warning**: ens indica els warnings o avisos que es produïsquen en manipular i realitzar operacions sobre la base de dades.
- **java.sql.driver**: esta interfície l'han d'implementar tots els fabricants de drivers que desitgen construir un driver JDBC. Representa un driver que podem utilitzar per a establir una connexió amb la base de dades.

- **java.sql.Types**: realitzen la conversió o mapatge de tipus estàndard del llenguatge SQL als tipus de dades del llenguatge Java.
- **java.sql.ResultSetMetaData**: esta interfície oferix informació detallada relativa a un objecte ResultSet determinat.
- **java.sql.DatabaseMetaData:** oferix informació detallada sobre la base de dades a la qual ens trobem connectats.

4 Treballar amb JDBC en un projecte

Per a utilitzar la funcionalitat de JDBC en un projecte Eclipsi cal descarregar el driver i configurar el projecte adequadament:

Descarregar el driver

Necessitem disposar del driver i saber exactament quin és el nom de la classe que el conté.

En el nostre cas, usarem bases de dades **MySQL**, per la qual cosa necessitem el driver JDBC de MySQL.

El driver es troba empaquetat en un fitxer **.jar** anomenat **MySQL-connector-java-version-bin**. Haurem de descarregar este fitxer al nostre equip des de la pàgina oficial del fabricador (MySQL).

Segons les especificacions que es poden llegir en la seua documentació la classe que conté el driver es diu **com.mysql.jdbc.driver**.

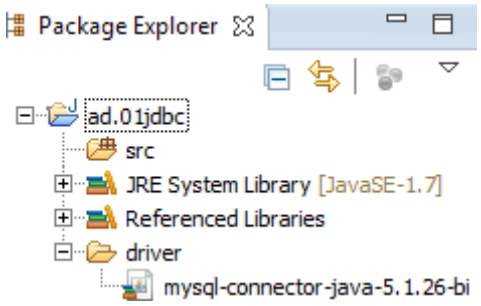
Crear el projecte i agregar el driver

Perquè l'aplicació funcione, el driver JDBC ha de ser present en l'equip del client.

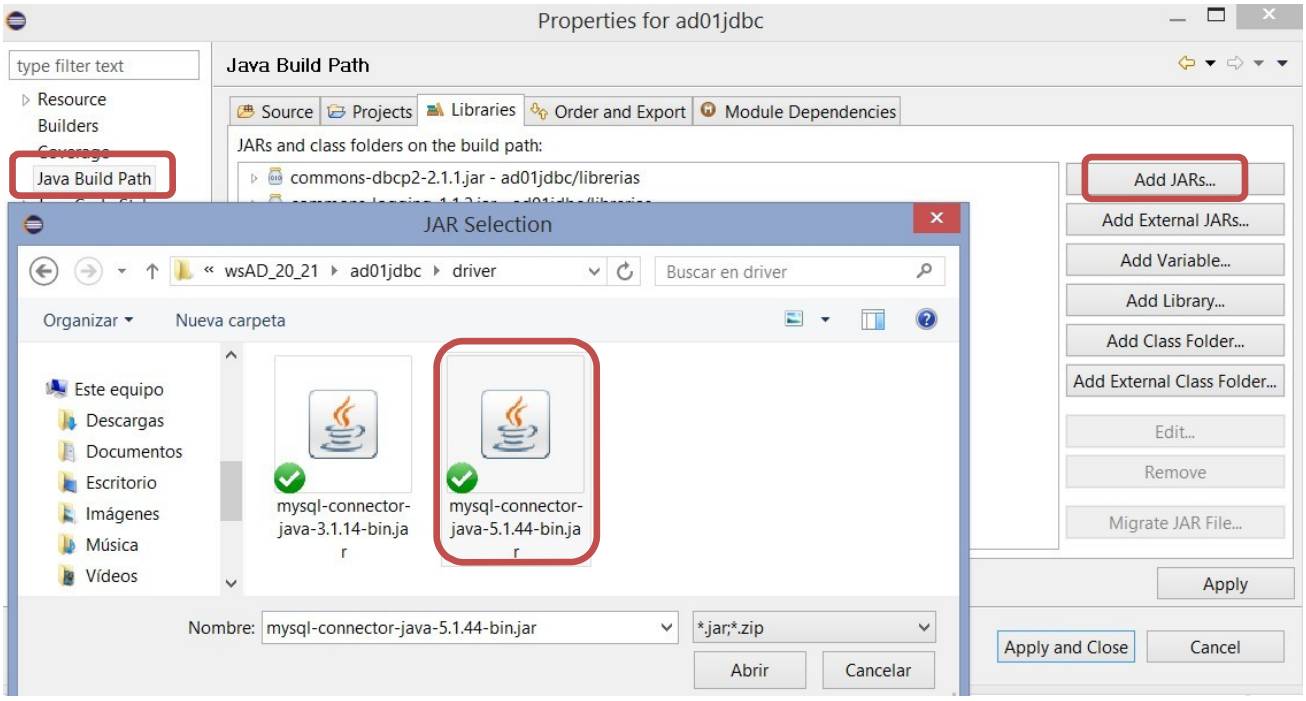
En Eclipse, quan s'exporta el projecte per a distribuir-lo al client, les llibreries utilitzades queden empaquetades convenientment i acompanyen al projecte. El que farem, per tant, és afegir el driver JDBC com a llibreria al projecte.

No és necessari que el driver JDBC estiga dins d'una carpeta del projecte per a incloure'l com a llibreria, però per facilitat a l'hora de treballar per a poder traslladar els projectes d'un ordinador a una altra de manera senzilla, situarem el **driver JDBC** en una **carpeta del projecte** al qual anomenarem **driver**.

1. Crea un projecte anomenat ad.01jdbc
2. Afig-li una carpeta (new – Folder), de nom **driver**.
3. Còpia en esta carpeta el fitxer **MySQL-connector-java-version-bin.jar** que has descarregat.



4. Agrega el driver al projecte com a biblioteca (library). Per a això:
    a. Accedix a la finestra de Propietats del projecte (botó dret sobre el projecte en el Package Explorer)
    b. En la finestra que s'obri, en l'arbre de l'esquerra triem "Java Build Path".
    c. En la part dreta de la finestra, seleccionem la pestanya "Libraries".
    d. Punxem sobre "Add JARs".
    e. Busquem en el disc el fitxer .jar que volem afegir i el seleccionem.
    f. Acceptar → Acceptar → Acceptar.



5 Connectar a BBDD amb JDBC usant DriverManager

Veurem com establir una connexió amb la base de dades utilitzant la classe driver Manager.

El codi de l'exemple complet és el següent:

```
public class _01ConectarConsDriverManager {
   static String DRIVER = "com.mysql.jdbc.Driver";
   static String URL = "jdbc:mysql://localhost:3306/ciclismo";
   static String USR = "root";
   static String PWD = "mysql";

   public static void main(String[] args) {
      Connection con = null;
      try{
         Class.forName(DRIVER).newInstance();
         System.out.println("Registrado");

         con = DriverManager.getConnection(URL, USR, PWD);
         System.out.println("Conectado");

      }catch (SQLException e){
         e.printStackTrace();

      }catch(Exception e){
         e.printStackTrace();
      }finally{
         try{
            if(con!=null && !con.isClosed())
               con.close();
               System.out.println("Desconectado");
         } catch(SQLException e){
            e.printStackTrace();
         }
      }
   }
}
```

1. Registrar el driver.

```
// new com.MySQL.JDBC.driver();
Class.forName(driver).newInstance();
```

La classe **java.sql.DriverManager** és el nivell o capa gestora del API JDBC, treballa entre l'usuari i els drivers. Té en compte els drivers disponibles i a partir d'ells establix una connexió entre una base de dades i el driver adequat per a eixa base de dades.

DriverManager manté una llista de classes Driver que han sigut registrades. Estes es poden registren utilitzant el mètode DriverManager.registerDriver(). No obstant això, les classes Driver estan escrites (o haurien d'estar-ho) de manera que es registren automàticament elles mateixes quan es crea l'objecte de la classe Driver. És a dir, que quan es carrega la classe del Driver, es crida automàticament a DriverManager.registerDriver(). No ha de fer-ho explícitament el programador.

Per a crear la classe no usarem l'operador new com és habitual. En canvi utilitzarem el mètode estàtic **forName(classe).newInstance(),** donant com a paràmetre el nom de la classe.

forName() permet crear una classe conegut el seu nom, dinàmicament, per programa. L'avantatge en este cas, enfront d'utilitzar l'operador new, radica en el fet que el nom de la classe podria estar contingut en un fitxer de configuració de l'aplicació. D'esta manera, si el nom del driver canviara no seria necessari redistribuir l'aplicació sinó, simplement, actualitzar el nom en este fitxer.

```
//Registrar el driver per a MySQL
String driver = "com.MySQL.JDBC.Driver";
Class.forName(driver).newInstance();

// Registrar driver per a Oracle
String driver = "oracle.JDBC.OracleDriver";
Class.forName(driver).newInstance();
```

Registrar el driver pot produir una sèrie d'excepcions, d'ací el bloc try-catch:

- ***forName()*** llança ***ClassNotFoundException*** si la classe no es troba.
- ***newInstance()*** llança ***InstantiationException*** si la classe o el constructor buit no són accessibles.
- ***newInstance****()* llança ***IllegalAccessException*** si es tracta d'una classe abstracta o un interfície.

2. Establir una connexió

```
String usr = "root";
String pwd = "MySQL";
String url= "JDBC:MySQL://localhost:3306/ciclisme";
Connection con = DriverManager.getConnection(url, usr, pwd);
```

Una vegada que les classes dels drivers s'han carregat i registrat per al DriverManager, estan disponibles per a establir una connexió amb una base de dades. La connexió es realitza mitjançant una cridada al mètode DriverManager.getConnection(), que retorna un objecte *Connection*.

Una sola aplicació pot tindre una o més connexions amb una mateixa base de dades, o pot tindre diverses connexions amb diferents bases de dades. ***Connection*** és una interfície, ja que com ja s'ha dit adés, JDBC oferix una plantilla o especificació que han d'implementar els fabricants de drivers de JDBC.

Per a poder connectar-se s'ha d'especificar la URL de la base de dades, per a això se seguix la següent nomenclatura:

jdbc:fabricant://servidor:port/id_basedades

jdbc:fabricant:@servidor:port:id_basedades

Per exemple l'URL per a una base de dades MySQL:

jdbc:mysql://localhost:3306/empresa

Per a Oracle:

jdbc:oracle:thin:@localhost:1521:pracs

3. Tancar la connexió.

Per a finalitzar, és important tancar totes les connexions obertes abans d'abandonar l'aplicació, perquè si no, el sistema gestor mantindrà en memòria la connexió malgastant recursos. Els objectes *Connection* disposen dels mètodes ***close***() i ***isClosed***(). El primer permet tancar la connexió, el segon permet determinar si la connexió continua oberta o ja ha sigut tancada.

El tancament de la connexió pot provocar una *SqlException*.

En resum, per a utilitzar una base de dades a través de JDBC hem de:

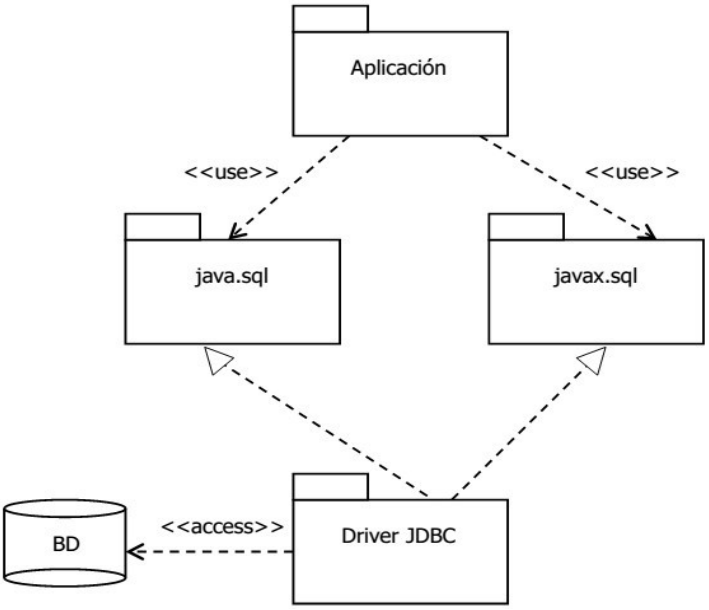
a) Afegir el driver al nostre projecte.

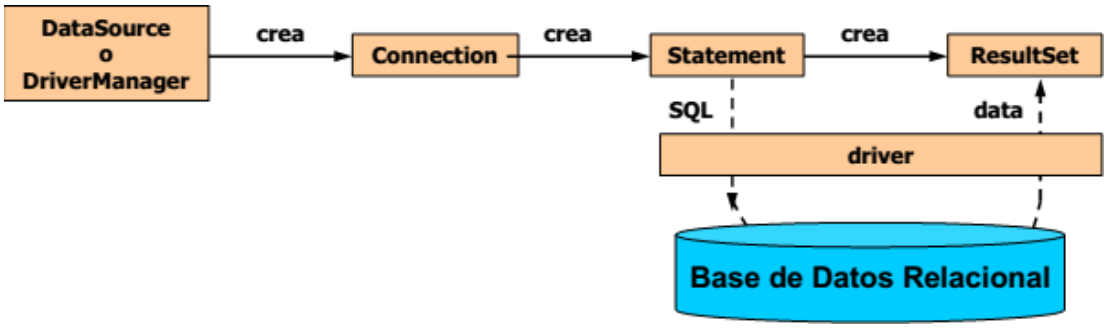
b) Registrar el driver.

c) Establir la connexió.

d) Realitzar operacions sobre la base de dades.

e) Tancar la connexió.





6 Connectar a BBDD amb JDBC usant DataSource

Hem vist com obtindre una connexió utilitzant *DriverManager.getConnection*(). No obstant això esta no és la forma que des de Java s'indica com més adequada.

L'establiment i tancament d'una connexió amb la base de dades és una tasca "pesada". Obrir una connexió, fer una tasca contra la base de dades i tancar la connexió pot ser lent. A vegades, potser, més que la pròpia acció que realitzem sobre la base de dades.

D'altra banda, mantindre una única connexió i compartir-la o reutilitzar-la, podria implicar problemes de concurrència de fils en l'accés a la connexió.

Per a evitar estos problemes s'utilitza ***DataSource***. *javax.sql.Datasource* és una interfície. ***DataSource.getConnection***() proporciona el mateix tipus de connexions que *DriverManager.getConnection*().

L'API de Java no proporciona cap classe que implemente esta interfície, per la qual cosa haurem de descarregar i utilitzar alguna implementació de tercers.

6.1 Tipus de DataSource

Un distribuïdor de programari podria proporcionar tres tipus de DataSources:

- **Implementacions bàsiques de *DataSource***, que no oferixen la possibilitat de *pooling* (agrupament) ni de transaccions distribuïdes.
- **Implementacions de *Datasource* que suporten *pooling* de connexions**: Produixen objectes Connection que es reutilitzen en lloc de tornar-se a crear cada vegada.
- **Implementacions de *Datasource* que suporten *pooling* de connexions i realització de transaccions distribuïdes**: Produixen objectes *Connection* que a més permeten la realització d'operacions que accedixen a més d'un SGBD.

6.2 Pool de connexions

En un DataSource que suporta *pooling* de connexions, la classe manté obertes diverses connexions a base de dades.

Quan algú necessita una connexió a base de dades, en comptes d'obrir-la directament amb *DriverManager.getConnection*(), li la demana al ***pool*** usant el seu mètode ***getConnection*()**. El *pool* agafa una de les connexions que ja té oberta, la marca com que algú l'està usant per a no donar-li-la a ningú més i la retorna. La següent cridada a este mètode getConnection(), buscarà una connexió lliure per a marcar-la com ocupada i la retornarà ... i així successivament.

Quan el que ha demanat la connexió acaba d'utilitzar-la, normalment després d'una transacció amb la base de dades o diverses seguides, crida al mètode ***Connection.close***(). Esta connexió que ens ha sigut entregada pel *pool*, realment no es tanca amb esta crida. El **mètode close()** únicament avisa al *pool* que ja ha acabat amb la connexió, de manera que sense tancar-la, la marca com a lliure per a poder entregar-li-la a un altre que ho demane

És molt important tancar les connexions per a què es puguen reutilitzar.

6.3 apache dbcp: BasicDataSource

commons-dbcp, d'Apache, és una implementació senzilla d'un *pool de connexions*. La classe ***BasicDataSource***.

Esta llibreria és **commons-dbcp** i la podem descarregar en el següent enllaç: http://commons.apache.org/proper/commons-dbcp/download_dbcp.cgi.

Esta llibreria necessita al mateix temps la llibreria **commons-pool**: http://commons.apache.org/proper/commons-pool/download_pool.cgi.

A més, també necessitarem la llibreria **commons-logging**: https://commons.apache.org/proper/commons-logging/download_logging.cgi.

Per a utilitzar-les en un projecte hem de descarregar-les i afegir-les a les llibreries, tal com hem fet amb el driver de JDBC.

Per a usar ***BasicDataSource*** no tenim més que fer un ***new*** d'eixa classe i passar-li els paràmetres adequats de la nostra connexió amb els mètodes ***setXXXXX()*** disponibles.

El següent exemple mostra com fer-ho:

```
public class _02ConectarConDataSource {
   final static String DRIVER = "com.mysql.jdbc.Driver";
   final static String URL = "jdbc:mysql://localhost:3306/ciclismo";
   final static String USR = "root";
   final static String PWD = "mysql";
   public static void main(String[] args) {
      Connection con = null;

      //Creamos datasource y configuramos sus parámetros.
      BasicDataSource ds = new BasicDataSource();
      ds.setDriverClassName(DRIVER);
      ds.setUrl(URL);
      ds.setUsername(USR);
      ds.setPassword(PWD);

      //Realizamos la conexión
      try {
         con = ds.getConnection();
         System.out.println("Conectado");
      } catch (SQLException e) {
         e.printStackTrace();
      }finally{
         //Cerramos la conexión
         try{
            if(con!=null && !con.isClosed())
               con.close();
               System.out.println("Desconectado");
         } catch(SQLException e){
            e.printStackTrace();
         }
      }

   }
}
```

També es podria utilitzar un bloc *try-catch with resources*, de manera que la connexió es tancaria automàticament. **Es proposa com a exercici**.

6.4 Fitxer de propietats

És possible crear una *DataSource* sense haver d'indicar per programa com és la url, el nom del driver, usuari, etc, sinó utilitzant un **fitxer de propietats**.

Disposar d'un fitxer de propietats presenta alguns avantatges: permetria, per exemple, modificar la ubicació de la base de dades que utilitza el programa ja en funcionament sense haver de modificar el programa. L'administrador del sistema del client, simplement hauria de modificar el fitxer de propietats.

Seguirem els següents passos:

1. Creem una carpeta (Folder) en el projecte de nom "**configuracion"**.
2. Dins de la carpeta "configuració", creem un fitxer de text anomenat **PropiedadesCiclismo.txt**, amb el següent contingut:

```
# Propietats per a la connexió a base de dades ciclismo
driverClassName=com.mysql.jdbc.driver
url=jdbc:mysql://localhost:3306/ciclismo
username=root
password=mysql
```

   En el fitxer especifiquem els valors per al driver, la url, etc. A l'esquerra del = apareix el nom de la propietat. Esta coincidix amb la paraula que usàvem en els mètodes *set* del *DataSource*, però començant en minúscula. És a dir, per a *setUrl*, la propietat serà *url*, per a *setPassword* serà *password*...

3. Per a accedir a les propietats:

```
Properties propietats = new Properties();

propietats.load(new FileInputStream("configuracion//
PropiedadesCiclismo.txt"));
```

   Properties és una classe descendent de HashTable, que es carrega amb parells (nom de la propietat, valor de la propietat) a través del mètode **load()**. ***Properties*** es pot carregar a partir d'un fitxer de text, però també a partir d'un fitxer xml.

4. Finalment, per crear el DataSource s'usa la classe B*asicDataSourceFactory*, de la següent manera:

```
DataSource ds = BasicDataSourceFactory.createDataSource(propietats);
```

Este seria el codi complet:

```
public class _03ConexionFicheroPropiedades {
    public static void main(String[] args) {
        Connection con = null;
        try{
            Properties propiedades = new Properties();
            propiedades.load(new FileInputStream("configuracion\\PropiedadesCiclismo.txt"));

            DataSource ds = BasicDataSourceFactory.createDataSource(propiedades);
            con = ds.getConnection();
            System.out.println("Conexion realizada");


    } catch(FileNotFoundException e){
        e.printStackTrace();
    } catch(IOException e){
        e.printStackTrace();
    } catch (Exception e){
        e.printStackTrace();
    }finally{
        //Cerramos la conexión
        try{
            if(con!=null && !con.isClosed())
                con.close();
                System.out.println("Desconectado");
        } catch(SQLException e){
            e.printStackTrace();
        }
    }
}
```

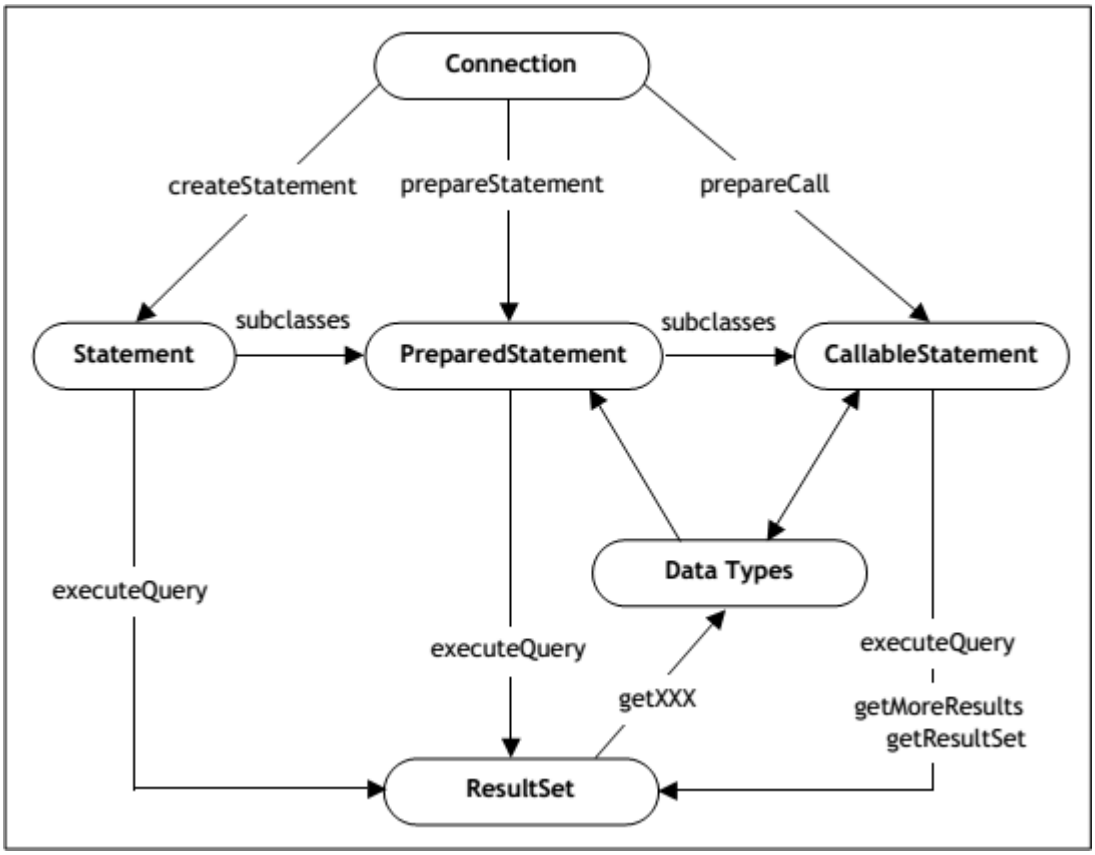
EXERCICI: Realitza un programa que connecte amb la base de dades agarrant les dades de la connexió d'un fitxer xml. (**_03ConexionFicheroXML**)

---

7 La interfície *Statement*

En el punt anterior véiem com establir una connexió amb una base de dades. En l'actual farem un pas més en l'accés a bases de dades a través de JDBC.

Una volta establida la connexió podrem enviar sentències SQL contra la base de dades a la qual ens hem connectat. Per a això comptem amb la interfície ***Statement***.



Un objecte ***Statement*** és utilitzat per a enviar **sentències SQL** a una base de dades. Existixen tres tipus d'objectes que representen una sentència SQL. Cadascun d'ells actua com un contenidor per a executar un tipus determinat de sentències SQL sobre una connexió a una base de dades.

Un objecte Statement es crea amb el mètode ***createStatement()*** de la classe ***Connection***, com es pot observar en el següent fragment de codi:

```
    Statement stmt = conexion.createStatement();
```

L'objecte Statement proporciona bàsicament tres mètodes, ***execute***(), ***executeUpdate***() i ***executeQuery***(), que actuen com conductors d'informació amb la base de dades. La diferència entre cadascun d'estos mètodes la podem vore en la següent taula:

Mètode	Ús recomanat
executeQuery()	S'utilitza amb sentències SELECT i retorna un ResultSet.
executeUpdate()	S'utilitza amb sentències INSERT, UPDATE i DELETE, o bé, amb sentències DDL SQL.
execute()	Utilitzat per a qualsevol sentència DDL, DML o comando específic de la base de dades.

Quan s'usa ***executeUpdate***():

- Per a instruccions d'inserció de tuples DDL: el valor retornat és sempre **zero**.
- Per executar instruccions de modificació de dades: Retornaran un valor major que zero, el qual és el **número de tuples afectades** en la relació.

En el següent fragment de codi es mostra com crear una taula i afegir registres utilitzant *executeUpdate*():

```
public class _04ExecuteUpdate {
   public static void main(String[] args) {
      Connection con = null;
      Statement stmt = null;
      try{
         Properties propiedades = new Properties();
         propiedades.load(new FileInputStream("configuracion\\PropiedadesCiclismo.txt"));

         DataSource ds = BasicDataSourceFactory.createDataSource(propiedades);
         con = ds.getConnection();
         System.out.println("Conexion realizada");

         //Creamos una tabla
         stmt = con.createStatement();
         String sql = "CREATE TABLE representante (" +
                              "id int(10) unsigned NOT NULL auto_increment," +
                              "nombre varchar(50) NOT NULL default ''," +
                              "fecha_ingreso date NOT NULL default '0000-00-00'," +
                              "salario float NOT NULL default '0'," +
                     "PRIMARY KEY (id))";
         stmt.executeUpdate(sql);
         System.out.println("Tabla creada");

         //Añadimos datos a la tabla representantes
         sql = "INSERT INTO representante VALUES (1,'pepe', '2005-01-01', 12000)";
         stmt.executeUpdate(sql);
         System.out.println("Inserciones realizadas");

      } catch(FileNotFoundException e){
         e.printStackTrace();
      } catch(IOException e){
         e.printStackTrace();
      } catch (Exception e){
         e.printStackTrace();
      }finally{
         //Cerramos el statement
         try{
            if(stmt !=null && !stmt.isClosed())
               stmt.close();
               System.out.println("Desconectado");
         } catch(SQLException e){
            e.printStackTrace();
         }
         //Cerramos la conexión
         try{
            if(con!=null && !con.isClosed())
               con.close();
               System.out.println("Desconectado");
         } catch(SQLException e){
            e.printStackTrace();
         }
      }
   }
}
```

En este altre exemple es realitza una consulta sobre la taula equip de la base de dades ciclisme per a determinar quants equips hi ha:

```
public class _05QueryNumeroEquipos {
    public static void main(String[] args) {
        Connection con = null;
        Statement stmt = null;
        ResultSet rs = null;
        try{
            Properties propiedades = new Properties();
            propiedades.load(new FileInputStream("configuracion\\PropiedadesCiclismo.txt"));

            DataSource ds = BasicDataSourceFactory.createDataSource(propiedades);

            con = ds.getConnection();
            System.out.println("Conexion realizada");

            // Realizamos la consulta
            stmt = con.createStatement();
            String sql = "SELECT count(*) FROM equipo";
            rs = stmt.executeQuery(sql);
            rs.first();
            System.out.println(rs.getInt(1));

        } catch(FileNotFoundException e){
            e.printStackTrace();
        } catch(IOException e){
            e.printStackTrace();
        } catch (Exception e){
            e.printStackTrace();
        }finally{
            //Cerramos el statement
            try{
                if(stmt !=null && !stmt.isClosed())
                    stmt.close();
                    System.out.println("Desconectado");
            } catch(SQLException e){
                e.printStackTrace();
            }
            //Cerramos el ResultSet
            try{
                if (rs!=null && !rs.isClosed()){
                    rs.close();
                    System.out.println("ResultSet cerrado");
                }
            }catch (SQLException e){
                e.printStackTrace();
            }
            //Cerramos la conexión
            try{
                if(con!=null && !con.isClosed())
                    con.close();
                    System.out.println("Desconectado");
            } catch(SQLException e){
                e.printStackTrace();
            }
        }
    }
}
```

Exemple INSERT

En este exemple hem posat les dades a inserir en unes matrius (arrays). No obstant, es poden recollir d'un altre lloc. Per exemple, si és una agenda de contactes, el millor és demanar-li'ls a l'usuari.

Ací estan a una matriu i amb un bucle els anem inserint. Fixeu-se que la sentència INSERT es va component a força de sumar cadenes. Seria més eficient amb un StringBuffer, però així se simplifica alguna cosa l'exemple. Noteu que en el INSERT fiquem cometes simples per als valors de cadenes, és a dir, en nom, cognoms i telèfon.

```
String noms[]={"Juan","Pedro","Antonio"};
String cognoms[]={"Gomez","Lopez","Alvarez"};
String telefons[]={"123","456","789"};

for (int i=0;i<nombres.length;i++) {
     stmt.executeUpdate("INSERT INTO contacte (nom, cognoms, telefon)
VALUES ('"+noms[i]+"','"+ cognoms[i]+"','"+telefons[i]+"' )");

}
```

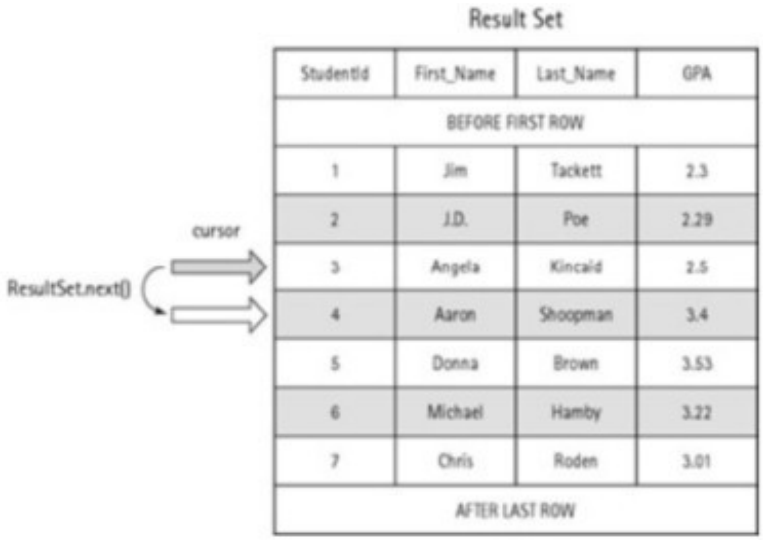
8 ResultSet

El resultat de l'execució del mètode *executeQuery*() és un objecte de tipus ResultSet.

Després de la consulta, l'objecte ResultSet conté les files resultat de l'execució de la sentència SQL. Este es pot recórrer per a recuperar les dades que conté, utilitzant una sèrie de mètodes que proporciona la classe.

L'aspecte que sol tindre un **ResultSet** és el d'una taula amb capçaleres de columnes i els valors corresponents retornats per una consulta en les files.

Un ResultSet manté un **cursor** que apunta a la **fila actual** de dades. El cursor es mou cap avall cada vegada que es diu al mètode ***next*().** Inicialment està posicionat **abans de la primera fila**, d'esta manera, la primera cridada a *next*() situarà el cursor en la primera fila, passant a ser la fila actual.



Cada crida successiva al mètode ***next*()** avança el cursor a la fila següent, fins que este se situe després de l'última fila.

A més d'este mètode de desplaçament bàsic, segons el tipus de ResultSet podrem realitzar desplaçaments lliures utilitzant mètodes com ***last*()**, ***first*()** o ***previous*()**, ...

Per a recuperar la informació que conté una fila s'utilitza una sèrie de mètodes ***getXXX*()** que retornen informació de les columnes de la fila actual.

Les files del ResultSet són retornades de dalt a baix segons es va desplaçant el cursor amb les successives cridades al mètode *next*(). **Un cursor és vàlid fins que l'objecte ResultSet o el seu objecte pare *Statement* es tanca**.

En el següent exemple s'il·lustra la manera de recórrer un ResultSet i recuperar informació per a mostrar els noms de tots els equips de la taula equips:

```
public class _06RecorridoBasicoResultSet {
      . . .
      . . .
      try{
         Properties propiedades = new Properties();
         propiedades.load(new FileInputStream("configuracion\\PropiedadesCiclismo.txt"));

         DataSource ds = BasicDataSourceFactory.createDataSource(propiedades);
         con = ds.getConnection();

         //Realizamos la consulta
         stmt = con.createStatement();
         String sql = "SELECT * FROM equipo";
         rs = stmt.executeQuery(sql);

         //Recorremos el ResultSet
         while (rs.next()){
            System.out.println(rs.getString(1));
         }

      } catch(FileNotFoundException e){

      . . .
      . . .
```

8.1 Necessitat d'alliberar els recursos

Les instàncies de *Connection* i les de *Statement* emmagatzemen en memòria molta informació relacionada amb les execucions realitzades. A més, mentre queden actives mantenen en el SGBD un conjunt important de recursos oberts, destinats a servir de manera eficient les peticions dels clients. El **tancament d'estos objectes permet alliberar recursos** tant del client com del servidor.

En realitat, els ***Statements*** són objectes vinculats íntimament a l'objecte *Connection* que els ha instanciat, i si no es tanquen específicament estaran actius mentre la connexió continue activa, fins i tot més enllà d'haver desaparegut la

variable que els referenciava. És més, la complexitat d'estos objectes fa que encara que s'haja tancat la connexió, els objectes *Statements* que no s'havien tancat expressament seguisquen més temps en memòria que els objectes tancats prèviament, ja que el *Garbage Collector* de Java haurà de fer més comprovacions per a assegurar que ja no disposa de dependències ni internes ni externes i es pot eliminar. És per això que es recomana sempre tancar-ho manualment utilitzant l'operació ***close()***.

El tancament dels objectes *Statement* assegura l'alliberament immediat dels recursos i l'anul·lació de les dependències. Si en un mateix mètode hem de tancar un objecte *Statement* i la connexió que l'ha instanciat, caldrà tancar en primer lloc el *Statement* i després la instància *Connection*. De fer-ho a l'inrevés, quan intentàrem tancar el Statement ens saltaria una excepció de tipus ***SqlException***, ja que el tancament de la connexió l'hauria deixada inaccessible.

A més de respectar l'ordre, cal assegurar l'alliberament dels recursos situant les operacions de tancament dins d'un bloc **finally**. D'esta manera, a pesar que es produïsquen errors, no es deixaran d'executar les instruccions de tancament.

Cal tindre en compte encara un detall més quan siga necessari realitzar el tancament de diversos objectes alhora. En este cas, a pesar que situàrem una darrere de l'altra totes les instruccions de tancament dins del bloc finally, no seria suficient garantia per a assegurar l'execució de tots els tancaments, ja que si mentre es produïx el tancament d'un dels objectes es llança una excepció, els objectes invocats en una posició posterior a la del qual s'ha produït l'error no es tancaran.

La solució d'este problema passa per evitar el llançament de qualsevol excepció durant el procés de tancament. Una possible forma és **encapsular cada tancament en un try-catch independent**, tal com hem vist en l'exemple anterior.

8.2 Tipus de ResultSet

El mètode **createStatement**() de la interfície **Connection** es troba sobrecarregat.

- Si utilitzem la seua versió **sense paràmetres** per a executar sentències SQL sobre l'objecte Statement que s'ha creat, s'obtindrà el tipus d'objecte ResultSet per defecte, és a dir, s'obtindria un tipus de cursor de només lectura i amb moviment únicament cap avant.
- Si optem pel mètode *createStatement*() amb **dos paràmetres** podrem definir el tipus d'objecte ResultSet que es retornarà com a resultat de l'execució d'una sentència SQL, com es mostra en el fragment de codi següent:

```
//Ús del mètode executeQuery
stmt = con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
ResultSet.CONCUR_READ_ONLY);
String sql = "select * from equip";
rs = stmt.executeQuery(sql);
```

1. El primer paràmetre indica el **tipus d'objecte ResultSet** que es crearà.
2. El segon paràmetre indica si el ResultSet és **només d'escriptura o si permet modificacions**, este paràmetre també es denomina **tipus de concurrència**.

Si especifiquem el tipus d'objecte ResultSet és obligatori indicar si serà de només lectura o no. Si no indiquem cap paràmetre en el mètode *createStatement*(), es crearà un objecte ResultSet amb els valors per defecte.

Els **tipus de ResultSet** diferents que es poden crear depenen del valor del primer paràmetre. Estos valors es corresponen amb constants definides en la interfície ResultSet. Estes **constants** es descriuen a continuació:

- **TYPE_FORWARD_ONLY:** es crea un objecte ResultSet amb moviment únicament cap avant (forward-only). És el tipus de ResultSet per defecte.
- **TYPE_SCROLL_INSENSITIVE:** es crea un objecte ResultSet que permet tot tipus de moviments. Però este tipus de ResultSet, mentre està obert, no serà

conscient dels canvis que es realitzen sobre les dades que està mostrant, i per tant no mostrarà estes modificacions.

- **TYPE_SCROLL_SENSITIVE:** igual que l'anterior permet tot tipus de moviments, i a més permet vore els canvis que es realitzen sobre les dades que conté.

Els valors que pot tindre el segon paràmetre que definix la creació d'un objecte ResultSet, són també **constants** definides en la interfície ResultSet i són les següents:

- **CONCUR_READ_ONLY:** indica que el ResultSet és només de lectura. És el valor per defecte.
- **CONCUR_UPDATABLE:** permet realitzar modificacions sobre les dades que conté el ResultSet.

8.3 Obtenint dades

Els mètodes getXXX() (getInt(), getString(), ...) oferixen els mitjans per a recuperar els valors de les columnes (camps) de la fila (registre) actual del ResultSet. Dins de cada columna no es necessita que les columnes siguen recuperades utilitzant un ordre determinat.

Per a designar una columna podem utilitzar:

- El seu nom.
- El seu número d'ordre.

Per exemple si la segona columna d'un objecte rs de la classe ResultSet es diu títol i emmagatzema dades de tipus String, es podrà recuperar el seu valor de les formes que mostra el següent fragment de codi:

```
String valor = rs.getString(2);
String valor = rs.getString("titule");
```

S'ha d'assenyalar que les columnes es numeren d'esquerra a dreta començant amb la columna 1, i que els noms de les columnes no són CASE SENSITIVE, és a dir, no distingixen entre majúscules i minúscules.

La informació referent a les columnes que conté el ResultSet es troba disponible cridant al mètode **getMetaData(),** este mètode retornarà un objecte ResultSetMetaData que contindrà el número, tipus i propietats de les columnes del ResultSet, més endavant veurem detalladament la interfície ResultSetMetaData.

Si coneixem el nom d'una columna, però no el seu índex, el mètode **findColumn**() pot utilitzar-se per a obtindre el número de columna, passant-li com a argument un objecte *String* que siga el nom de la columna corresponent, este mètode ens retornarà un enter que serà l'índex corresponent a la columna.

8.4 Tipus de dades i conversions

Quan es llança un mètode **getXXX**() determinat sobre un objecte ResultSet per a obtindre el valor d'un camp del registre actual, el driver JDBC convertix la informació que es vol recuperar al tipus Java especificat i retorna un valor Java adequat. Per exemple si utilitzem el mètode getString() i el tipus de la dada en la base de dades és VARCHAR, el driver JDBC convertirà el VARCHAR a un objecte String de Java, per tant el valor de retorn de getString() serà un objecte de la classe String.

Esta conversió de tipus es pot realitzar gràcies a la classe **java.sql.Types**. En esta classe es definixen el que es denominen **tipus de dades JDBC**, que es correspon amb els tipus de dades SQL estàndard. Això ens permet abstraure'ns del tipus SQL específic de la base de dades amb la qual estem treballant, ja que els tipus JDBC són tipus de dades SQL genèrics.

Normalment estos tipus genèrics ens serviran per a totes les nostres aplicacions JDBC. La **classe Types** està definida com un conjunt de constants (final static). Estes constants s'utilitzen per a identificar els tipus SQL. Si el tipus de la dada SQL que tenim en la nostra base de dades és específic d'eixa base de dades, i no es troba entre les constants definides per la classe Types, s'utilitzarà el tipus **Types.OTHER**.

La correspondència de tipus entre la base de dades i Java es mostra en la següent taula:

TIPUS SQL	TIPUS JAVA
BIT(1)	java.lang.Boolean
BIT(> 1)	byte[]
TINYINT	java.lang.Boolean if the configuration property "tinyInt1isBit" is set to "true" (the default) and the storage size is "1", or java.lang.Integer if not.
BOOL	BOOLEAN
SMALLINT[(M)] [*UNSIGNED]	java.lang.Integer
MEDIUMINT[(M)] [*UNSIGNED]	java.lang.Integer
INT,INTEGER[(M)] [UNSIGNED]	java.lang.Integer, if UNSIGNED java.lang.Long
BIGINT[(M)] [UNSIGNED]	java.lang.Long , if UNSIGNED java.math.BigInteger
FLOAT[(M,D)]	java.lang.Float
DOUBLE[(M,B)]	java.lang.Double
DECIMAL[(M[,D])	java.math.BigDecimal
DATE	java.sql.Date (**No confondre amb java.Util.Date**)
DATETIME	java.sql.Timestamp
TIMESTAMP[(M)]	java.sql.Timestamp
TIME	java.sql.Time
YEAR[(2|4)]	java.sql.Date
CHAR(M)	java.lang.String (unless the character set for the column is binary , then byte[] is returned.
VARCHAR(M) [BINARY]	java.lang.String (unless the character set for the column is binary, then byte[] is returned.
BINARY(M)	byte[]
VARBINARY(M)	byte[]
TINYBLOB	byte[]
TINYTEXT	java.lang.String
BLOB	byte[]
TEXT	java.lang.String
MEDIUMBLOB	byte[]

MEDIUMTEXT	`java.lang.String`
TIPUS SQL	TIPUS JAVA
LONGBLOB	`byte[]`
LONGTEXT	`java.lang.String`
ENUM('value1','value2',...)	`java.lang.String`
SET('value1','value2',...)	`java.lang.String`

A continuació es mostra una taula resumida amb els mètodes getXXX() i el seu corresponent valor retornat:

	TINYINT	SMALLINT	INTEGER	BIGINT	REAL	FLOAT	DOUBLE	DECIMAL	NUMERIC	BIT	CHAR
getByte	X	x	x	x	x	x	x	x	x	x	x
getShort	x	X	x	x	x	x	x	x	x	x	x
getInt	x	x	X	x	x	x	x	x	x	x	x
getLong	x	x	x	X	x	x	x	x	x	x	x
getFloat	x	x	x	x	X	x	x	x	x	x	x
getDouble	x	x	x	x	x	X	X	x	x	x	x
getBigDecimal	x	x	x	x	x	x	x	X	X	x	x
getBoolean	x	x	x	x	x	x	x	x	x	X	x
getString	x	x	x	x	x	x	x	x	x	x	X
getBytes											
getDate											x
getTime											x
getTimestamp											x
getAsciiStream											x
getUnicodeStream											x
getBinaryStream											
getObject	x	x	x	x	x	x	x	x	x	x	x

	LONGVARCHAR	BINARY	VARBINARY	LONGVARBINARY	DATE	TIME	TIMESTAMP	VARCHAR
getByte	x	x						
getShort	x	x						
getInt	x	x						
getLong	x	x						
getFloat	x	x						
getDouble	x	x						
getBigDecimal	x	x						
getBoolean	x	x						
getString	X	x	x	x	x	x	x	x
getBytes			X	X	x			
getDate	x	x				X		x
getTime	x	x					X	x
getTimestamp	x	x				x	x	X
getAsciiStream	x	X	x	x	x			
getUnicodeStream	x	X	x	x	x			
getBinaryStream			x	x	X			
getObject	x	x	x	x	x	x	x	x

On hi ha una **x** minúscula s'indica la **possibilitat de fer-ho** i amb **X** s'indica que és l'opció **més recomanable**.

8.5 Desplaçaments

En este apartat comentarem les diferents formes que tenim de desplaçar-nos dins de un objecte ResultSet, depenent del tipus de ResultSet. També vorem en este apartat cóm mostrar els continguts de cada registre (fila) d'un objecte ResultSet utilitzant els mètodes getXXX().

Com ja s'havia nomenat anteriorment, el mètode **next()** de la interfície ResultSet l'utilitzarem per a desplaçar-nos al registre següent dins d'un ResultSet.

El mètode **next()** retorna un valor booleà (tipus Boolean de Java), true si el registre següent existix i false si hem arribat al final de l'objecte ResultSet, és a dir, no hi ha més registres. Este era l'únic mètode que oferia la interfície ResultSet en la versió 1.0 de JDBC, però en JDBC 2.0, a més de tindre el mètode next(), disposem dels següents **mètodes per al desplaçament i moviment dins d'un objecte ResultSet**:

- **boolean absolute(int registre):** desplaça el cursor al nombre de registres indicat. Si el valor és negatiu, es posiciona en el número registre indicat però

començant pel final. Este mètode retornarà false si ens hem desplaçat després de l'últim registre o abans del primer registre de l'objecte ResultSet.

Per a poder utilitzar este mètode l'objecte ResultSet ha de ser de tipus TYPE_SCROLL_SENSITIVE o de tipus TYPE_SCROLL_INSENSITIVE. A un ResultSet que és de qualsevol d'estos dos tipus es diu que és de tipus SCROLLABLE. Si a este mètode li passem un valor zero es llançarà una excepció SqlException.

- **void afterLast():** es desplaça al final de l'objecte ResultSet, després de l'últim registre. Si el ResultSet no posseïx registres este mètode no tenen cap efecte. Este mètode només es pot utilitzar en objectes ResultSet de tipus SCROLLABLE, com succeïx amb molts dels mètodes comentats en estos punts.
- **void beforeFirst():** mou el cursor al començament de l'objecte ResultSet, abans del primer registre. Només es pot utilitzar sobre objectes ResultSet de tipus SCROLLABLE.
- **boolean first():** desplaça el cursos al primer registre. Retorna true si el cursor s'ha desplaçat a un registre vàlid, per contra, retornarà false en un altre cas o bé si l'objecte ResultSet no conté registres. Igual que els mètodes anteriors, només es pot utilitzar en objectes ResultSet de tipus SCROLLABLE.
- **boolean last():** desplaça el cursor a l'últim registre de l'objecte ResultSet. Retornarà true si el cursor es troba en un registre vàlid, i false en un altre cas o si l'objecte ResultSet no té registres. Només és vàlid per a objectes ResultSet de tipus SCROLLABLE, en cas contrari llançarà una excepció SqlException.
- **void moveToCurrentRow():** Després d'una inserció, mou el cursor a la posició en què es trobava abans de la inserció. Este mètode només té sentit quan estem situats dins del ResultSet en un registre que s'ha inserit. Este mètode només és vàlid utilitzar-lo amb objectes ResultSet que permeten la modificació, és a dir, estan definits mitjançant la constant CONCUR_UPDATABLE.
- **boolean previous():** desplaça el cursor al registre anterior. És el mètode contrari al mètode next(). Retornarà true si el cursor es troba en un registre o fila vàlids, i false en cas contrari. Només és vàlid este mètode amb objectes ResultSet de tipus SCROLLABLE, en cas contrari llançarà una excepció SqlException.

- **boolean relative(int registres):** mou el cursor un nombre relatiu de registres, este número pot ser positiu o negatiu. Si el número és negatiu el cursor es desplaçarà cap al principi de l'objecte ResultSet el nombre de registres indicats, i si és positiu es desplaçarà cap al final d'objecte ResultSet corresponent. Este mètode només es pot utilitzar si el ResultSet és de tipus SCROLLABLE.

També existixen altres mètodes dins de la interfície ResultSet que estan relacionats amb el desplaçament:

- **boolean isAfterLast():** indica si ens trobem després de l'últim registre de l'objecte ResultSet. Només es pot utilitzar en objectes ResultSet de tipus SCROLLABLE.
- **boolean isBeforeFirst():** indica si ens trobem abans del primer registre de l'objecte ResultSet. Només es pot utilitzar en objectes ResultSet de tipus SCROLLABLE.
- **boolean isFirst():** indica si el cursor es troba en el primer registre. Només es pot utilitzar en objectes ResultSet de tipus SCROLLABLE.
- **boolean isLast():** indica si ens trobem en l'últim registre del ResultSet. Només es pot utilitzar en objectes ResultSet de tipus SCROLLABLE.
- **int getRow():** retorna el número de registre actual. El primer registre serà el número 1, el segon el 2, etc. Retornarà zero si no hi ha registre actual.

8.6 Mètodes UpdateXXX ()

Per a poder modificar les dades que conté un ResultSet hem de crear un **ResultSet de tipus modificable.** Per a això hem d'utilitzar la constant **ResultSet.CONCUR_UPDATABLE** dins del mètode *createStatement()*.

Un ResultSet que permet modificacions sol permetre diferents **desplaçaments** (se sol utilitzar la constant **ResultSet.TYPE_SCROLL_INSENSITIVE** o **ResultSet.TYPE_SCROLL_SENSITIVE**) però no és del tot necessari ja que també pot ser del tipus "només cap avant" (**forward-only**).

Per a modificar els valors d'un registre existent s'utilitzen una sèrie de mètodes **updateXXX()** de la interfície ResultSet. Les XXX indiquen el tipus de la dada igual que ocorre amb els mètodes getXXX() d'esta mateixa interfície.

El procés per a realitzar la modificació d'una fila d'un ResultSet és el següent:

1. Ens situem sobre el registre que volem modificar.
2. Llancem els mètodes updateXXX() adequats, passant-li com a argument els nous valors.
3. Llancem el mètode updateRow() perquè els canvis tinguen efecte sobre la base de dades.

El mètode **updateXXX()** rep dos paràmetres, el **camp** o columna a modificar i **el nou valor**. La columna la podem indicar pel seu número d'ordre o bé pel seu nom, igual que en els mètodes getXXX().

El següent fragment de codi mostra com es pot modificar el camp direcció de l'últim registre d'un ResultSet que conté el resultat d'una SELECT sobre la taula de clients:

```
Statement stmt =
con.createStatement( ResultSet.TYPE_SCROLL_SENSITIVE,
ResultSet.CONCUR_UPDATABLE);
//Executem la SELECT sobre la taula clients
String sql = "select * from ciclistas";
rs = stmt.executeQuery(sql);
System.out.println("Situem el cursor al final");
```

```
rs.last();
rs.updateString("dorsal", 125);
rs.updateRow();
```

Si ens desplacem dins del ResultSet abans de llançar el mètode updateRow(), es perdran les modificacions realitzades. I si volem cancel·lar les modificacions llançarem el mètode **cancelRowUpdates()** sobre l'objecte ResultSet, en lloc del mètode updateRow().

Una vegada que hem invocat el mètode updateRow(), el mètode cancelRowUpdates() no tindrà cap efecte.

El mètode **cancelRowUpdates**() cancel·la les modificacions de tots els camps d'un registre, és a dir, si hem modificat tres camps amb el mètode updateXXX() es cancel·laran les tres modificacions.

8.7 Mètodes insertRow() i deleteRow()

A més de poder realitzar modificacions directament sobre les files d'un ResultSet, també podem afegir noves files (registres) i eliminar les existents. Per a fer-ho, els mètodes corresponents són: **moveToInsertRow**() i **deleteRow**().

El primer pas per a inserir un registre o fila en un ResultSet és moure el cursor (punter que indica el registre actual) del ResultSet, això s'aconseguix mitjançant el mètode moveToInsertRow().

El següent pas és donar un valor a cadascun dels camps que formaran part del nou registre, per a això s'utilitzen els mètodes updateXXX() adequats.

Per a finalitzar el procés es llança el mètode insertRow(), que crearà el nou registre tant en el ResultSet com en la taula de la base de dades corresponents. Fins que no es llança el mètode insertRow(), la fila no s'inclou dins del ResultSet. És una fila especial denominada "fila d'inserció" (insertrow) i és similar a un buffer completament independent de l'objecte ResultSet.

El següent fragment de codi dona d'alta un nou registre en la taula de clients:

```
Statement stmt = conn.createStatement(
 ResultSet.TYPE_SCROLL_SENSITIVE,ResultSet.CONCUR_UPDATABLE);
//Executem la SELECT sobre la taula clients
String sql = "select * from clients";
rs = stmt.executeQuery(sql);
//Creem un nou registre en la taula de clients
rs.moveToInsertRow();
rs.updateString(2,"Joan Boix");
rs.updateString(3,"Major 36");
rs.insertRow();
```

Si no facilitem valors a tots els camps del nou registre amb els mètodes updateXXX(), eixe camp tindrà un valor NULL, i si en la base de dades no està definit eixe camp per a admetre nuls es produirà una excepció **SqlException**.

Quan hem inserit el nostre nou registre en l'objecte ResultSet, **podrem tornar a l'antiga posició en la qual ens trobàvem dins del ResultSet**, abans d'haver llançat el

mètode *moveToInsertRow*(), cridant al mètode ***moveToCurrentRow***(), este mètode només es pot utilitzar en combinació amb el mètode *moveToInsertRow*().

A continuació tenim un exemple de com s'**elimina una fila**. En primer lloc cal situar-nos sobre la fila que es vol suprimir. Després executarem el mètode ***deleteRow***().

```
Statement stmt = conn.createStatement(
 ResultSet.TYPE_SCROLL_SENSITIVE,ResultSet.CONCUR_UPDATABLE);
//Executem la SELECT sobre la taula clients
String sql = "select * from clients";
rs = stmt.executeQuery(sql);
//Ens situem al final del ResultSet
rs.last();
rs.deleteRow();
```

9 Objectes PreparedStatement

Esta interfície, igual que la interfície Statement, ens permet executar sentències SQL sobre una connexió establida amb una base de dades. Però en este cas executarem sentències SQL més especialitzades. Estes sentències SQL es diran **sentències SQL precompilades** i rebran paràmetres d'entrada.

La interfície **PreparedStatement** hereta de la interfície Statement i es diferencia d'ella de dues maneres:

- Les instàncies de *PreparedStatement* contenen **sentències SQL que ja han sigut compilades**. Això és el que fa a una sentència "prepared" (preparada).
- La sentència SQL que conté un objecte *PreparedStatement* pot contindre un o més paràmetres d'entrada. Un **paràmetre d'entrada** és aquell el valor el qual no s'especifica quan la sentència és creada, en el seu lloc la sentència tindrà un signe d'interrogació **(?)** per cada paràmetre d'entrada.

Abans d'executar-se la sentència s'ha d'especificar un valor per a cadascun dels paràmetres a través dels mètodes **setXXX**() apropiats. Estos mètodes setXXX() els afig la interfície *PreparedStatement*.

Pel fet que les sentències dels objectes *PreparedStatement* estan precompilades **la seua execució serà més ràpida que la dels objectes Statement**. Per tant, una sentència SQL que serà executada diverses vegades se sol crear com un objecte *PreparedStatement* per a guanyar en eficiència. També s'utilitzarà este tipus de sentències per a passar-li paràmetres d'entrada a les sentències SQL.

En heretar de la interfície *Statement*, la interfície *PreparedStatement* hereta totes funcionalitats de *Statement*. A més, afig una sèrie de mètodes que permeten assignar un valor a cadascun dels paràmetres d'entrada d'esta mena de sentències.

Els mètodes **execute**(), **executeQuery**() i **executeUpdate**() són sobrecarregats i en esta versió, els objectes *PreparedStatement* no prenen cap tipus d'arguments. Per tant, a estos mètodes mai se'ls haurà de passar per paràmetre l'objecte String que representava la sentència SQL a executar. En este cas, un objecte *PreparedStatement* ja és una sentència SQL per si mateixa, a diferència del que ocorria amb les sentències Statement que posseïen un significat només en el moment en el qual s'executaven.

Per a crear un objecte PreparedStatement s'ha de llançar el mètode **prepareStatement**() de la interfície **Connection** sobre l'objecte que representa la connexió establida amb la base de dades .

En el següent fragment de codi es pot vore com es crearia un objecte *PreparedStatement* que representa una sentència SQL amb dos paràmetres d'entrada. L'objecte *pstmt* contindrà la sentència SQL indicada, la qual, ja ha sigut enviada al DBMS i ja ha sigut preparada per a la seua execució. En este cas s'ha creat una sentència SQL amb dos paràmetres d'entrada.

```
conn = DriverManager.getConnection(JDBCUrl,"root","");
pstmt = conn.prepareStatement("update factures set serie=?
 where id=?");
```

Els **interrogants** indiquen els paràmetres que han d'assignar-se a esta sentència SQL.

9.1 Utilització paràmetres d'entrada

Abans de poder executar un objecte *PreparedStatement* s'ha d'assignar un valor per a cadascun dels seus paràmetres. Això es realitza cridant a un mètode **setXXX**(), on XXX és el tipus apropiat per al paràmetre. Per exemple, si el paràmetre és de tipus long, el mètode a utilitzar serà setLong().

El **primer argument** dels mètodes setXXX() és la **posició** ordinal del **paràmetre** al qual se li assignarà valor, i el **segon argument** és el **valor** a assignar.

Per exemple, en el següent fragment de codi es crea un objecte *PreparedStatement* i tot seguit li assigna al primer dels seus paràmetres un valor de tipus String i al segon un valor de tipus Long.

```
String JDBCUrl = "JDBC:MySQL://localhost:3306/empresa";
conn = DriverManager.getConnection(JDBCUrl,"root","");
pstmt = conn.prepareStatement("update factures set serie=?
 where id=?");
pstmt.setString(1, "B");
pstmt.setLong(2, 1);
pstmt.executeUpdate();
```

Una vegada que s'ha **assignat** uns **valors** als **paràmetres d'entrada** d'una sentència, l'objecte *PreparedStatement* es pot executar múltiples vegades, fins que siguen esborrats els paràmetres amb el mètode **clearParameters**().

No obstant això, no és necessari cridar a este mètode quan es vulguen modificar els paràmetres d'entrada, sinó que en llançar els nous mètodes **setXXX**() els valors dels paràmetres seran reemplaçats.

Per acabar, es crida al mètode **executeUpdate()** de l'objecte *PreparedStatement* i d'esta manera es voran reflectits els canvis en la base de dades.

10 Transaccions

En alguns moments podem necessitar que diverses sentències SQL s'executen com un tot, i si cap falla d'elles es desfacen els canvis realitzats per les sentències i es torne a la situació anterior.

Per exemple, en realitzar una transferència d'un compte d'un banc a un altre compte, les dues sentències encarregades de realitzar l'actualització en el saldo de cada compte s'han d'executar, i si una d'elles falla s'han de desfer els canvis realitzats. Estes dues sentències s'han d'executar dins d'una mateixa **transacció**.

Per a que la transacció es duga a terme, totes les sentències s'han d'executar amb èxit. Si alguna d'elles falla, es desfan els canvis que s'hagen pogut realitzar per a evitar qualsevol tipus d'inconsistència en la base de dades.

Les transaccions permeten preservar la integritat de les dades i es sol dir que dins d'una transacció es dona un **"tot o res"**.

Quan establim una connexió a una base de dades amb JDBC, per defecte es crea en mode autocommit. Això vol dir que cada sentència SQL modifica la base de dades només executar-se, és a dir, es tracta com una transacció que conté una única sentència.

Si volem agrupar diverses sentències SQL en una mateixa transacció utilitzarem el mètode **setAutoCommit()** de la interfície Connection, passant-li com a paràmetre el valor **false**, per a indicar que una sentència no s'execute automàticament.

Una vegada que s'ha deshabilitat el mode autocommit, les sentències que executem no modificaran la base de dades fins que no cridem al mètode **commit**() de la interfície Connection. Totes les sentències que s'han executat prèviament a la crida del **mètode commit()** s'inclouran dins d'una mateixa transacció i es duran a terme sobre la base de dades com un tot.

Reprenent l'exemple de la transferència entre dos comptes, i suposant que el número de compte del *compte d'origen* de la transferència és el 1, i el de destí el 2 el següent codi efectuaria estes dues modificacions dins d'una transacció:

```
try{
  int transfe=5000;
```

```
      int cuentaOrigen=1; int
      cuentaDestino=2;
      conexion.setAutoCommit(false);


String          sql  =  “UPDATE  Comptes  SET  quantitat=?  WHERE
NCuenta=?”

      PreparedStatement sentenResta=conexion.prepareStatement();
      sentenResta.setInt(1,transfe);
      sentenResta.setInt(2,cuentaOrigen);
      sentenResta.executeUpdate(sql);


      Statement sentenciaSuma=conexion.createStatement();
      sentenciaSuma.executeUpdate("UPDATE Comptes SET quantitat=
      quantitat+"+transfe+"WHERE Compte="+cuentaDestino);


      conexion.commit();
      conexion.setAutoCommit(true);


    }catch(SqlException ex){ conexion.rollback();
    System.out.println("La transacció ha fallat.");
    conexion.setAutoCommit(true);
    }
```

En este cas si tot va bé (no es produïx cap excepció) **la transferència es realitza quan es crida al mètode commit()** de la interfície Connection.

En cas de produir-se algun error, s'avorta la transacció mitjançant el **mètode rollback()**, també de la interfície Connection.

11 Maneig d'errors en JDBC

Una **bona gestió d'errors** és una de les tasques més importants quan s'està desenvolupant una aplicació. S'ha de tindre en compte cadascun dels possibles errors que pot causar el sistema per a donar la màxima seguretat de les dades amb els quals l'aplicació està treballant.

Quan s'està treballant amb JDBC es pot donar el cas que existisquen errors, també anomenats en el llenguatge Java ***Exceptions***. Interactuar amb la Base de dades és un procés complex en el qual intervenen molts components i tots ells han de garantir que el sistema funcione a la perfecció.

Quan algun component d'estos falla donarà una **Exception** que hem de capturar-la en la nostra aplicació per a tractar de resoldre el problema.

Ací vam mostrar una llista xicoteta d'algunes **àrees que poden ocasionar problemes**:

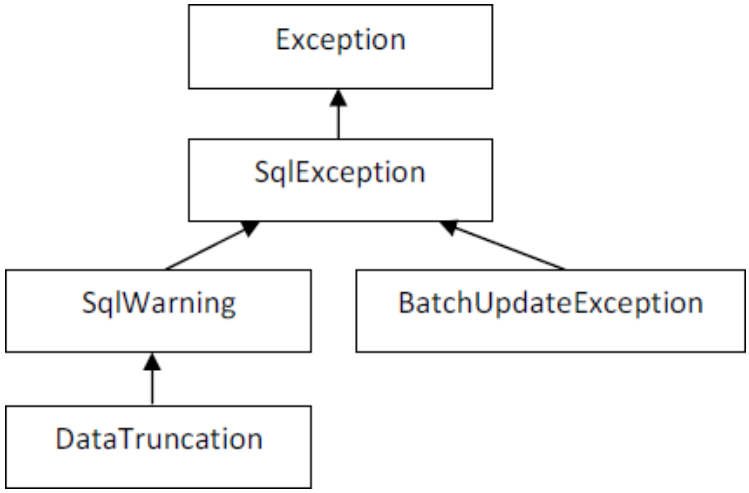
- La Connectivitat en la xarxa
- Drets d'accés i autenticació d'usuaris
- Sentències SQL invàlides
- Errors deguts a la càrrega dels drivers
- Conversions entre tipus invàlides
- Etc...

Tots estos problemes i molts més poden ser susceptibles de llançar una excepció. A més, quasi tots els mètodes JDBC llancen una SqlException per si ocorre algun error. En altres paraules, que **tota sentència en JDBC ha d'anar acompanyada dels blocs try-catch o propagar l'excepció al mètode que l'ha invocat**.

Quan treballem amb JDBC podem trobar 4 tipus d'excepcions:

- La classe **SqlException** hereta de Exception i per tant té tota la funcionalitat per a manejar excepcions Java.
- La classe **SQLWarning** i **BatchUpdateException** hereten de SqlException.
- La classe **DataTruncation** hereta de SQLWarning.

La següent figura mostra la relació entre cadascuna d'estes classes:



11.1 SqlException

Quasi tots els mètodes del API JDBC llancen una SqlException. Normalment quan estem programant l'accés a través de JDBC, enquadrem el codi JDBC en **blocs try-catch** com es pot vore en el següent fragment de codi:

```
try{
   //Procés JDBC....
}catch(SqlException es){
   //Excepció produïda per JDBC
   es.printStackTrace();
}catch(Exception e){
  //Una altra excepció
   e.printStackTrace();
}
```

La classe **SqlException** especialitza a la classe Exception, **afegint molta més informació acosta possibles errors**. La taula que vam mostrar a continuació mostra la informació més útil que podem consultar quan ha ocorregut una SqlException:

INFORMACIÓ	EXPLICACIÓ
Error message	Cadena que conté una breu descripció de l'error que s'ha produït.
Error code	Codi específic del venedor. Depén de la base de dades que hi

	haja per davall.
SQLState	Codi d'error estandarditzat, basat en l'especificació X/Open SQL.
Cause	Una SqlException pot estar relacionada amb una o diverses causes que no són més que un o més objectes Throwable que van provocar el llançament de la SqlException. Amb getCause() es poden recuperar estos objectes de forma recursiva, fins que getCause() retorne null.

El següent fragment de codi mostra com podem obtindre informació quan es produïx una SqlException. En este cas s'està intentant consultar els valors d'una taula que no existix en la base de dades.

```
public static void main(String[] args) {
 …
 try{
   …
   rs = stmt.executeQuery("select * from taula_noExistix");
 }catch(SqlException es){
   System.out.println("Code: " + es.getErrorCode());
   System.out.println("sqlState: " + es.getSQLState());
   System.out.println("Error Message: " + es.getMessage());
   es.printStackTrace();
   Throwable t = es.getCause();
   while(t != null) {
     System.out.println("Cause: " + t);
     t = t.getCause();
   }
 }catch(Exception e){
   e.printStackTrace();
 } //end try
 …
}
```

Pot ser convenient realitzar un processament de les excepcions en funció del seu codi d'error.

11.2SQLWarning

La classe SQLWarning una subclasse de SqlException. Que es produïsca un SQLWarning no ocasiona la parada del programa, però sí que informa de possibles anomalies. Podria produir-se un warning si, per exemple, tractem de modificar els permisos d'un usuari i no s'aconseguix.

Els principals objectes que llancen SQLWarning són **Connection**, **Statement** i **ResultSet**. Estes classes tenen un mètode **getWarnings()** que retorna el primer dels warnings que s'han produït sobre l'objecte (pot haver-se produït més d'un). La resta de warnings es recuperen recursivament a partir del primer.

Exemple:

```
Statement stmt =con.executeUpdate(…);
SQLWarning w = stmt.getWarnings();
while (w != null) {
 System.out.println("Message: " + w.getMessage());
 System.out.println("sqlState: " + w.getSQLState());
 System.out.print("Vendor error code: ");
 System.out.println(w.getErrorCode());
 System.out.println("");
 w = w.getNextWarning();
}
```

Per exemple, una possible situació que llance un esdeveniment SQLWarning pot ser que intentem crear un objecte ResultSet "*updateable*" però que el nostre driver no ho permeta. Això provocarà que es llance un esdeveniment SQLWarning indicant que el nostre driver no suporta ResultSet "*updateables*" i treballarà en manera normal, però l'aplicació seguirà el seu curs.

11.3 DataTruncation

L'excepció **DataTruncation** ocorre quan intentem escriure o recuperar alguna dada de la base de dades que per alguna raó la longitud de la qual ha sigut truncada.

Per exemple, este error pot ocórrer si modifiquem la propietat ***Statement.setMaxFieldSize***() amb un valor xicotet.

Normalment quan es produïx un **DataTruncation** sobre una lectura de dades sobre la BD se sol llançar un SQLWarning silenciós. No obstant això, si estem **realitzant operacions d'escriptura** normalment es produïx una **SqlException**.

De tota manera estes implementacions depenen molt del **fabricant del driver**, així que el millor és tirar mà de la **documentació sobre el maneig d'errors**.

12 DatabaseMetadata

Esta interfície ens permet obtindre gran quantitat d'**informació sobre la base de dades** a la qual ens hem connectat. Tracta a la base de dades com un tot.

Per a obtindre esta informació, esta interfície aporta un gran número de mètodes diferents. Molts d'ells retornen objectes **ResultSet** contenint la informació corresponent, per tant, haurem d'usar els mètodes getXXX() per a recuperar la informació dels ResultSet.

Estos **mètodes** els podem dividir en:

- Mètodes que retornen una informació senzilla, tipus de Java.
- Mètodes que retornen una informació més complexa, com pot ser un objecte ResultSet.

Algunes de les capçaleres d'estos mètodes prenen com a paràmetre patrons de cadenes, en els quals es poden utilitzar caràcters comodí, de manera que només les dades que es corresponguen amb el patró donat seran retornats pel mètode llançat:

- "%" identifica una cadena de 0 o més caràcters.
- "_" s'identifica amb un sol caràcter.

Si un driver no suporta un mètode de la interfície DatabaseMetaData es llançarà una excepció **SqlException**, i en el cas que el mètode retorne un objecte ResultSet, s'obtindrà un ResultSet buit.

Per a obtindre un objecte ***DatabaseMetaData*** sobre el qual llançar els mètodes que ens donaran la informació sobre el SGBD, s'ha de llançar el mètode **getMetaData**() sobre l'objecte **Connection** que es corresponga amb la connexió a la BD de la qual volem obtindre la informació. Ho podem observar en el següent fragment de codi:

```
Connection      con  = DriverManager.getConnection(JDBCUrl,"root",
"");
  DatabaseMetadata dbmd = conn.getMetaData();
```

12.1 Obtenint informació d'una Base de dades

La interfície **DatabaseMetadata** ens oferix una informació detallada sobre la base de dades. Vorem alguns d'estos mètodes a través d'un exemple.

Realitzarem una aplicació Java que es connecte a una base de dades de MySQL. Una volta connectats obtindrem algunes dades d'interés sobre ella. Els **mètodes** que s'utilitzen d'esta interfície s'han dividit en **cinc mètodes diferents**, atesa la informació que obtenim a través d'estos: **producte**, **driver**, **funcions**, **taules** i **procediments**.

En el següent fragment de codi podem veure algun d'estos mètodes:

```
Connection con= null;
try{
   //Connexió -------------
   Properties props = new Properties();
   props.load(new FileInputStream("PropiedadesCiclismo.txt"));
   DataSource                          ds                          =
BasicDataSourceFactory.createDataSource(props);
   con = ds.getConnection();
   System.out.println("Connexió realitzada");

   //DatabaseMetadata
   DatabaseMetaData dbmd = con.getMetaData();

   //Informació de la base de dades
   infoProducte(dbmd);

   //Informació del driver
   infoDriver(dbmd);

   //Informació de les funcions suportades pel SGBD
   infoFuncions(dbmd);

   //Informacion de les taules, vistes, etc ...
   infoTaules(dbmd);
```

```
   //Informacion dels proc. emmagatzemats
   infoProcediments(dbmd);
} catch (Exception e){
    e.printStackTrace();
} finally {
    try{
        if(con != null) con.close();
    } catch (SqlException e){
        e.printStackTrace();
    }
}
```

infoProducte()

El primer dels mètodes és el mètode **infoProducte**(). Este mètode ens oferix informació general sobre el sistema gestor de base de dades: el seu *nom*, *versió*, *URL de JDBC*, *usuari* connectat, *característiques que suporta*, etc.

A continuació anem mostrar el fragment de com s'ha realitzat el mètode:

```
public static void infoProducte(DatabaseMetaData dbmd)
    throws SqlException{
   //Nom del SGBD
   String producte = dbmd.getDatabaseProductName();
   //Versió del SGBD
   String versio = dbmd.getDatabaseProductVersion();
   //Usuari connectat
   String nomUsr = dbmd.getUserName();
   //url de la base de dades
   String url = dbmd.getURL();
   System.out.println();
   System.out.println("------ INFORMACIO SOBRE EL DBMS----------");
   System.out.println("Base dades: " + producte);
```

```
    System.out.println("Versió: " + versio);
    System.out.println("Usuari actual " + nomUsr);
    System.out.println("URL: " + url);
}
```

infoDriver()

El següent mètode que es llança és el mètode **infoDriver**(). Dins d'este mètode s'obtenen informació sobre el driver que s'utilitza per a realitzar la connexió, la informació que s'obté és: el nom del driver, versió, versió inferior i superior:

```
public static void infoDriver(DatabaseMetaData dbmd) throws
SqlException{
    String driver = dbmd.getDriverName();
    String driverVersion = dbmd.getDriverVersion();
    System.out.println();
    System.out.println("---- INFORMACIO SOBRE EL driver---------");
    System.out.println("driver: " + driver);
    System.out.println("Versió: " + driverVersion);
}
```

infoFuncions()

La informació sobre les funcions que oferix la nostra base de dades l'obtenim a partir del mètode **infoFuncions**(). Este mètode ens indicarà les funcions que suporta el nostre sistema gestor de base de dades pel que fa a cadenes de caràcters, funcions numèriques, data i hora i funcions del sistema.

```
public static void infoFuncions(DatabaseMetaData dbmd) throws
SqlException{
    String funcCadenes = dbmd.getStringFunctions();
    String funcSistema = dbmd.getSystemFunctions();
    String funcTemps = dbmd.getTimeDateFunctions();
    String funcNumeriques = dbmd.getNumericFunctions();
    System.out.println();
    System.out.println("---- FUNCIONS QUE SUPORTA EL DBMS---------");
```

```
    System.out.println(" Funcions de Cadenes: "+funcCadenes);
    System.out.println(" Funcions Numèriques: "+funcNumeriques);
System.out.println("           Funcions        del        Sistema:
"+funcionesSistema);
System.out.println("        Funcions     de     Data     i     Hora:
"+funcionesTemps);
}
```

infoTaules()

En els mètodes vistos fins ara, els mètodes que s'han utilitzat de la interfície DatabaseMetaData retornen valors que es corresponen amb objectes String o tipus int de Java. En este nou mètode, que ens retorna informació sobre les taules de la base de dades, la informació es retorna dins d'un objecte ResultSet amb un format determinat.

La crida al mètode **getTables**() és més complicada que les cridades a la resta dels mètodes de la interfície DatabaseMetaData vistos fins ara:

- Els dos primers paràmetres del mètode getTables() s'utilitzen per a obtindre les taules d'un catàleg, en el nostre cas no utilitzarem esta opció i per tant li passem un null.
- El segon paràmetre, l'esquema , el passarem també a null.
- El tercer paràmetre és el **patró de cerca** que s'aplicarà al nom de les taules. Si utilitzem el caràcter especial % obtindrem totes les taules.
- L'últim dels paràmetres indica el tipus de taula que volem obtindre. Este paràmetre és un array amb tots els tipus de taules de les quals volem obtindre informació. Els tipus de taula són: **TABLE, VIEW, SYSTEM TABLE, GLOBAL TEMPORARY, LOCAL TEMPORARY, ALIAS** i **SYNONYM**.

En este exemple es vol recuperar informació sobre les taules d'usuari i de sistema, per la qual cosa només tindrem un array amb dos elements, el primer d'ells contindrà la cadena TABLE i el segon la cadena SYSTEM TABLE.

En l'objecte ResultSet retornat pel mètode getTables() trobarem en cada fila del mateix informació sobre cada taula. Este ResultSet retornat té un format determinat, cada columna conté una informació concreta de la taula que es correspon amb la fila actual del ResultSet.

Els noms d'estes columnes són:

- **TABLE_CAT**: catàleg de la taula.
- **TABLE_SCHEM**: esquema de la taula.
- **TABLE_NAME**: nom de la taula.
- **TABLE_TYPE**: tipus de la taula.
- **REMARKS**: comentaris sobre la taula.

Per a recuperar qualsevol columna d'este ResultSet utilitzarem el mètode getString() de la interfície ResultSet, ja que tota la informació que conté el ResultSet és de tipus cadena de caràcters.

```
public static void infoTaules(DatabaseMetaData dbmd) throws
 SqlException{
   ResultSet rsTaules = dbmd.getTables(null,null,null,
    new String[]{"TABLE","SYSTEM_TABLE"});
   System.out.println();
   System.out.println("------ TAULES----------");
   while(rsTaules.next())
       { System.out.println(rsTaules.getString("TABLE_CAT"));
       System.out.println(rsTaules.getString("TABLE_SCHEM"));
       System.out.println(rsTaules.getString("TABLE_NAME"));
       System.out.println(rsTaules.getString("TABLE_TYPE"));
       System.out.println(rsTaules.getString("REMARKS"));
       System.out.println("-------------------");
   }
 }
```

infoProcediments()

El mètode **infoProcediments()** és molt similar a l'anterior. En lloc de recuperar informació sobre les taules de la base de dades, obtindrem informació sobre els procediments emmagatzemats de la base de dades.

```
 public static void infoProcediments(DatabaseMetaData dbmd) throws
SqlException{
```

```
    ResultSet rsProced = dbmd.getProcedures(null,null,null);
    System.out.println();
System.out.println("------                                    PROCEDIMENTS
EMMAGATZEMATS");

while(rsProced.next())
{ System.out.println(rsProced.getString("PROCEDURE_CAT"));
System.out.println(rsProced.getString("PROCEDURE_SCHEM"))
;

System.out.println(rsProced.getString("PROCEDURE_NAME"));
System.out.println(rsProced.getString("PROCEDURE_TYPE"));
System.out.println(rsProced.getString("REMARKS"));
System.out.println("");
}
}
```

El **mètode getProcedures()** de la interfície DatabaseMetaData també retorna un objecte ResultSet. Els paràmetres són els mateixos que en getTables(), a excepció del array de String que, en este cas, no es contempla.

Els **noms de les columnes** del ResultSet retornat pel mètode **getProcedures**() són:

- **PROCEDURE_CAT**: catàleg del procediment.
- **PROCEDURE_SCHEM**: esquema del procediment.
- **PROCEDURE_NAME**: nom del procediment.
- **REMARKS**: comentaris sobre el procediment.
- **PROCEDURE_TYPE**: tipus de procediment.

13 ResultSetMetadata

Esta interfície té una finalitat molt semblant a la vista en el punt anterior. La interfície ResultSetMetaData ens oferix una sèrie de mètodes **que ens permeten obtindre informació sobre les columnes que conté el ResultSet**. És una interfície més particular que l'anterior:

- Un objecte DatabaseMetaData és comú a tota la base de dades-
- Cada objecte **ResultSetMetaData** és particular per a cada ResultSet.

Un objecte ResultSetMetaData l'obtindrem llançant el mètode **getMetaData() sobre l'objecte ResultSet** sobre el qual vulguem consultar la informació de les seues columnes.

La interfície ResultSetMetaData proporciona mètodes per a recuperar informació del ResultSet. La quantitat de mètodes que oferix esta interfície és molt de menor a la que oferia la interfície DatabaseMetaData. A continuació vam mostrar una taula amb els més usats:

Mètode	Descripció
getTableName()	Retorna un String amb el nom de la taula que s'ha consultat en el ResultSet.
getColumnCount()	Retorna un int amb el número de columnes que s'ha consultat en el ResultSet
getColumnName(int n)	Retorna un String amb el nom de la columna en la posició n
getColumnType(int n)	Retorna el tipus JDBC de la columna en la posició n
getColumnTypeName(int n)	Retorna el nom del tipus JDBC que es correspon amb el tipus de la columna en la posició n.

A continuació mostrarem un fragment de codi en el qual, usant ResultSetMetaData, es consultarà la taula port de la base de dades ciclisme i es mostrarà informació de cadascuna de les columnes: nom de la columna, el tipus i altra informació rellevant.

```
...
Statement stm = con.createStatement();
ResultSet rs = stm.executeQuery("SELECT * FROM port");
// DatabaseMetadata
ResultSetMetaData rsmd = rs.getMetaData();
int numColumnas = rsmd.getColumnCount();
for(int i = 1; i<=numColumnas ;i++){
   System.out.println("Columna " + i);
   System.out.println("Nom: " + rsmd.getColumnName(i));
   System.out.println("Tipus: " + rsmd.getColumnType(i));
   System.out.println("Tipus: " + rsmd.getColumnTypeName(i));
   System.out.println("Display Size: " +
    rsmd.getColumnDisplaySize(i));
   System.out.println("ClassName:                " +
rsmd.getColumnClassName(i));
   System.out.println("Label: " + rsmd.getColumnLabel(i));
   System.out.println("Precisió: " + rsmd.getPrecision(i));
   System.out.println("Escala: " + rsmd.getScale(i));
   System.out.println("Nullable: " + rsmd.isNullable(i));
   System.out.println("Autoincrement: "+rsmd.isAutoIncrement(i));
   System.out.println("-----------------");
}
...
```

14 Objectes CallableStatement

L'últim tipus de sentències que podem utilitzar en JDBC són les sentències **CallableStatement**. Esta interfície hereta de la interfície PreparedStatement i oferix la possibilitat de manejar paràmetres d'eixida i de realitzar cridades a procediments emmagatzemats de la base de dades. Estes cridades les fa d'una forma estàndard per a tots els SGBD.

Un procediment emmagatzemat es troba dins d'una base de dades. La cridada a un procediment és el que conté un objecte CallableStatement. Esta crida està escrita amb una sintaxi d'escapament que pot tindre dues formes diferents:

- **Amb un paràmetre de resultat:** és un tipus de paràmetre d'eixida que representa el valor retornat pel procediment. La **sintaxi** de crida és la següent: **`{?=callnom_del_procediment[(?,?...)]}`**

- **Sense cap paràmetre de resultat i amb paràmetres d'entrada:** La sintaxi seria la següent:

  `{callnom_del_procediment[(?,?,...)]}`

La **interrogació** (?) representarà al paràmetre, bé siga d'entrada o d'eixida.

- **Sense cap paràmetre de resultat i ni paràmetres d'entrada** La sintaxi d'una seria:

  `{callnom_del_procediment}`

La interfície CallableStatement hereta els mètodes de la interfície Statement, que s'encarreguen de sentències SQL generals, i també els mètodes de la interfície PreparedStatement, que s'encarreguen de manejar paràmetres d'entrada.

14.1 Utilització de paràmetres

Com hem vist adés, una sentència CallableStatement pot tindre paràmetres d'entrada, d'eixida i d'entrada/eixida. Per a passar-li paràmetres d'entrada a un objecte CallableStatement, s'utilitzen els mètodes **setXXX**() que heretava de la interfície PreparedStatement.

Si el procediment emmagatzemat retorna **paràmetres d'eixida,** el tipus JDBC de cada paràmetre d'eixida ha de ser registrat abans d'executar l'objecte CallableStatement corresponent.

Per a registrar els tipus JDBC dels paràmetres d'eixida s'ha de llançar el mètode **CallableStatement.registerOutParameter()**.

Després d'executar la sentència, es poden recuperar els valors d'estos paràmetres cridant al mètode **getXXX**() adequat. El mètode getXXX() ha de recuperar el tipus Java que es correspondria amb el tipus JDBC amb el qual es va registrar el paràmetre. Als mètodes getXXX() se li passarà un enter que indicarà el valor ordinal del paràmetre a recuperar.

En el següent fragment de codi es registra un paràmetre d'eixida, executa el procediment cridat per l'objecte CallableStatement i després recupera el valor dels paràmetres d'eixida.

```
String JDBCUrl = "jdbc:mysql://localhost:3306/empresa";
Connection conn = DriverManager.getConnection(JDBCUrl,"root","");

//Preparem la crida al procediment
CallableStatement cstmt = conn.prepareCall("{call comprar(?,?)}");

cstmt.registerOutParameter(2, java.sql.TYPEs.Integer);
cstmt.setInt(1, 360);
cstmt.execute();
```

15 JDBC Batch Processing

La interfície Statement també suporta el processament de múltiples sentències DML en manera batch. D'esta manera minimitzem el número de cridades a la base de dades, el qual a voltes és crític en segons quines aplicacions.

Per exemple, suposem una aplicació que necessite realitzar moltes sentències de tipus INSERT i de tipus UPDATE per a controlar un determinat sistema. Si no utilitzem processament batch podem sobrecarregar en excés amb cridades a la BD i disminuir el rendiment general de l'aplicació. Per a això si utilitzem el processament batch podem agrupar diverses sentències INSERT i UPDATE i llançar-les cap a la Base de dades en una única crida.

Per poder treballar d'esta manera, és necessari primer saber si la nostra **base de dades suporta el processament Batch**. Podem consultar-ho amb la interfície DatabaseMetaData i cridar al mètode supportBatchUpdates(): si retorna true vol dir que la nostra BD suporta processament batch.

En el nostre cas sí que tenim suport batch.

```
  //Comprovar suport batch
  Connection                          conn                          =
DriverManager.getConnection(JDBCUrl,"root","");
DatabaseMetaData dm = conn.getMetaData();
System.out.println("Suporta procesament batch ->
"+dm.supportsBatchUpdates());
```

Per a **utilitzar el processament batch** sobre la interfície Statement cridem al **mètode addBatch()** que rep com a paràmetre la sentència SQL i este procés el repetim per a cadascuna de les sentències SQL que vulguem afegir en batch.

Una volta tinguem totes les sentències, cridem al **mètode executeBatch()** de la interfície Statement. Prèviament hem d'haver desactivat el autocommit amb el mètode setAutoCommit() i finalment invocar al mètode **commit**() per a fer els canvis.

En el següent exemple es creen 3 articles nous en manera batch sobre la base de dades empresa en MySQL:

```
   Connection                          conn                          =
DriverManager .getConnection(JDBCUrl,"root","");


   conn.setAutoCommit(false);
   Statement stmt = conn.createStatement();


   //Afegim sentències SQL a mode Batch

   String sql = "insert into articu values(8,'HD 120G',100.0)";
   stmt.addBatch(sql);
   sql = "insert into articu values(9,'HD 160G',120.0,)";
   stmt.addBatch(sql);
   sql = "insert into articu values(10,'HD 200G',140.0)";
   stmt.addBatch(sql);
   int result[] = stmt.executeBatch();
   conn.commit();
```

15.1 BatchUpdateException

Els objectes Statement, PreparedStatement i CallableStatement llancen excepcions BatchUpdateException si ocorre un error quan s'està treballant en manera batch.

Esta classe hereta de SqlException i per tant conté la mateixa informació. Però a més a més, afig propietats específiques que t'ajuden a determinar quina ha sigut la sentència que ha provocat l'error.